ANS 1.

Solution: We apply the KDE formula:

$$p(1.5) = \frac{1}{3}\left[K\left(\frac{1.5-1}{1}\right) + K\left(\frac{1.5-2}{1}\right) + K\left(\frac{1.5-3}{1}\right)\right]$$

Now, we calculate each term using the Gaussian kernel:

- $K\left(\frac{1.5-1}{1}\right) = K(0.5) = \frac{1}{\sqrt{2\pi}}e^{-\frac{0.5^2}{2}} = \frac{1}{\sqrt{2\pi}}e^{-0.125} \approx 0.3521$
- $K\left(\frac{1.5-2}{1}\right) = K(-0.5) = \frac{1}{\sqrt{2\pi}}e^{-\frac{(-0.5)^2}{2}} = \frac{1}{\sqrt{2\pi}}e^{-0.125} \approx 0.3521$ (The Gaussian kernel is symmetric)
- $K\left(\frac{1.5-3}{1}\right) = K(-1.5) = \frac{1}{\sqrt{2\pi}}e^{-\frac{(-1.5)^2}{2}} = \frac{1}{\sqrt{2\pi}}e^{-1.125} \approx 0.1295$

Finally, we average these contributions:

$$p(1.5) = \frac{1}{3}(0.3521 + 0.3521 + 0.1295) \approx \frac{0.8337}{3} \approx 0.2779$$

ANS 2.

Solution: In 2D, we use a *multivariate* Gaussian kernel:

$$K(\mathbf{u}) = \frac{1}{2\pi}e^{-\frac{\|\mathbf{u}\|^2}{2}}$$

where $\mathbf{u}$ is a 2D vector, and $\|\mathbf{u}\|$ is its Euclidean norm (length). The KDE formula becomes:

$$p(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} K\left(\frac{\mathbf{x}-\mathbf{x}_i}{h}\right)$$

Here, $\mathbf{x}$ and $\mathbf{x}_i$ are 2D vectors. Let's calculate the density at $\mathbf{x} = (2,1)$:

- For $\mathbf{x}_1 = (1,1)$: $\mathbf{u} = \frac{(2,1)-(1,1)}{1} = (1,0)$, $\|\mathbf{u}\|^2 = 1^2 + 0^2 = 1$, $K = \frac{1}{2\pi}e^{-0.5} \approx 0.0965$
- For $\mathbf{x}_2 = (2,2)$: $\mathbf{u} = \frac{(2,1)-(2,2)}{1} = (0,-1)$, $\|\mathbf{u}\|^2 = 0^2 + (-1)^2 = 1$, $K \approx 0.0965$
- For $\mathbf{x}_3 = (3,1)$: $\mathbf{u} = \frac{(2,1)-(3,1)}{1} = (-1,0)$, $\|\mathbf{u}\|^2 = (-1)^2 + 0^2 = 1$, $K \approx 0.0965$

Averaging the contributions:

$$p(2,1) = \frac{1}{3}(0.0965 + 0.0965 + 0.0965) = \frac{0.2895}{3} \approx 0.0965$$

ANS 3.

First, calculate the Gaussian probabilities for $x = 2$:

- For component 1:

$$\mathcal{N}(2|\mu_1 = 1, \sigma_1^2 = 1) = \frac{1}{\sqrt{2\pi \cdot 1}} e^{-\frac{(2-1)^2}{2 \cdot 1}} = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}} \approx 0.2420$$

- For component 2:

$$\mathcal{N}(2|\mu_2 = 5, \sigma_2^2 = 1) = \frac{1}{\sqrt{2\pi \cdot 1}} e^{-\frac{(2-5)^2}{2 \cdot 1}} = \frac{1}{\sqrt{2\pi}} e^{-\frac{9}{2}} \approx 0.0044$$

Next, compute the posterior probabilities (responsibilities) for $x = 2$:

- $p(z_1 = 1|x = 2) = \frac{\pi_1 \mathcal{N}(2|\mu_1, \sigma_1^2)}{\pi_1 \mathcal{N}(2|\mu_1, \sigma_1^2) + \pi_2 \mathcal{N}(2|\mu_2, \sigma_2^2)} = \frac{0.5 \cdot 0.2420}{0.5 \cdot 0.2420 + 0.5 \cdot 0.0044} \approx \frac{0.1210}{0.1210 + 0.0022} \approx 0.9821$
- $p(z_2 = 1|x = 2) = \frac{\pi_2 \mathcal{N}(2|\mu_2, \sigma_2^2)}{\pi_1 \mathcal{N}(2|\mu_1, \sigma_1^2) + \pi_2 \mathcal{N}(2|\mu_2, \sigma_2^2)} = \frac{0.5 \cdot 0.0044}{0.1210 + 0.0022} \approx 0.0179$

To update μ_1, compute responsibilities for all data points:

- For $x = 1$:

$$\mathcal{N}(1|1, 1) \approx 0.3989, \quad \mathcal{N}(1|5, 1) \approx 0.0001$$

$$p(z_1 = 1|1) = \frac{0.5 \cdot 0.3989}{0.5 \cdot 0.3989 + 0.5 \cdot 0.0001} \approx 0.9997$$

- For $x = 5$:

$$\mathcal{N}(5|1, 1) \approx 0.0073, \quad \mathcal{N}(5|5, 1) \approx 0.3989$$

$$p(z_1 = 1|5) = \frac{0.5 \cdot 0.0073}{0.5 \cdot 0.0073 + 0.5 \cdot 0.3989} \approx 0.0180$$

- For $x = 6$:

$$\mathcal{N}(6|1, 1) \approx 5.55 \times 10^{-6}, \quad \mathcal{N}(6|5, 1) \approx 0.2420$$

$$p(z_1 = 1|6) = \frac{0.5 \cdot 5.55 \times 10^{-6}}{0.5 \cdot 5.55 \times 10^{-6} + 0.5 \cdot 0.2420} \approx 0.000023$$

$$\mu_1 = \frac{\sum_{i=1}^{n} \gamma(z_{i1}) x_i}{\sum_{i=1}^{n} \gamma(z_{i1})} = \frac{1 \cdot 0.9997 + 2 \cdot 0.9821 + 5 \cdot 0.0180 + 6 \cdot 0.000023}{0.9997 + 0.9821 + 0.0180 + 0.000023}$$

- Numerator: $0.9997 + 1.9642 + 0.0900 + 0.000138 \approx 3.0540$
- Denominator: $0.9997 + 0.9821 + 0.0180 + 0.000023 \approx 1.9998$
- $\mu_1 \approx \frac{3.0540}{1.9998} \approx 1.527$

Using the original (possibly approximate) values:

- Numerator: $1 \cdot 0.9997 + 2 \cdot 0.9821 + 5 \cdot 0.0179 + 6 \cdot 0.0044 \approx 3.0798$
- Denominator: $0.9997 + 0.9821 + 0.0179 + 0.0044 \approx 2.0041$
- $\mu_1 \approx \frac{3.0798}{2.0041} \approx 1.537$

ANS 4.

Using Euclidean distance: $d(P_1, P_2) = \sqrt{[(x_2-x_1)^2 + (y_2-y_1)^2]}$

	A	B	C	D	E
A	0	1.00	1.00	7.21	8.06
B	1.00	0	1.41	6.40	7.21
C	1.00	1.41	0	7.81	8.60
D	7.21	6.40	7.81	0	1.00
E	8.06	7.21	8.60	1.00	0

Step 2: Agglomerative Clustering Steps

Initial: C = {{A}, {B}, {C}, {D}, {E}}

Iteration 1:

- Best pair: (A, B) or (A, C) with distance 1.00. Let's choose (A, B)
- Merge: C_m ← MERGE(A, B) = {A, B}
- Update: C ← {{A, B}, {C}, {D}, {E}}
- Distance from {A,B} to others (using single-linkage):
    - d({A,B}, C) = min(d(A,C), d(B,C)) = min(1.00, 1.41) = 1.00
    - d({A,B}, D) = min(d(A,D), d(B,D)) = min(6.71, 6.40) = 6.40
    - d({A,B}, E) = min(d(A,E), d(B,E)) = min(7.81, 7.21) = 7.21

Iteration 2:

- Best pair: ({A, B}, C) with distance 1.00
- Merge: C_m ← MERGE({A,B}, C) = {A, B, C}
- Update: C ← {{A, B, C}, {D}, {E}}
- Distance from {A,B,C} to others:
    - d({A,B,C}, D) = min(d(A,D), d(B,D), d(C,D)) = 6.40
    - d({A,B,C}, E) = min(d(A,E), d(B,E), d(C,E)) = 7.21

Iteration 3:

- Best pair: (D, E) with distance 1.00
- Merge: C_m ← MERGE(D, E) = {D, E}
- Update: C ← {{A, B, C}, {D, E}}

ANS 5.

1. Approach Type:

This is a **divisive** approach (top-down). We can tell because:

- We start with all points in one huge cluster
- In every step, one current cluster is split into two smaller clusters
- The hierarchy moves from a single cluster toward individual points

2. Key Characteristic:

Hierarchical clustering is **rigid in nature**: once a cluster is made (or split), it cannot be undone. In this example:

- Once we split $\{P_1, P_2, P_3, P_4, P_5, P_6\}$ into $\{P_1, P_2, P_3\}$ and $\{P_4, P_5, P_6\}$, this decision is permanent
- When we split $\{P_1, P_2, P_3\}$ into $\{P_1, P_2\}$ and $\{P_3\}$, we cannot reconsider the previous split or merge P_3 with points from $\{P_4, P_5, P_6\}$
- This rigidity means there are less chances of improvement compared to iterative methods

3. Stopping Point:

We would stop **after split 2**, which gives us exactly 3 clusters: $\{\{P_1, P_2\}, \{P_3\}, \{P_4, P_5, P_6\}\}$

This uses the **stopping criterion based on quantity**: stop when the number of clusters equals k (where k = 3 in this case).

ANS 6.

a) Starting Point:

- Agglomerative (Bottom-Up): Initially each point is a cluster itself. Starts with n clusters for n data points.
- Divisive (Top-Down): Initially all points are in one huge cluster. Starts with 1 cluster containing all data.

b) Direction of Hierarchy Building:

- Agglomerative: Generates a bottom-up hierarchy. In every step, two clusters are merged into one, reducing the number of clusters.
- Divisive: Generates a top-down hierarchy. In every step, one current cluster is split into two, increasing the number of clusters.

c) Fixed Number of Clusters: Both approaches do not require a specific number of clusters. Hierarchical methods create a hierarchy of clusters (multi-level partitions) and return a set of nested clusters. You can:

- Stop when clustering is good enough using a stopping criterion
- Stop based on quantity when the number of clusters equals a desired k
- Generate the full hierarchy and cut at any level

ANS 7.

a) Quality vs Quantity:

Stopping based on quality measures (diameter, radius, density) rather than just quantity allows us to:

- Ensure clusters meet certain cohesiveness standards (e.g., points are close together within clusters)
- Avoid creating clusters that are too loose or spread out
- Find the natural grouping structure in the data rather than forcing an arbitrary number of clusters
- Take advantage of the hierarchical method's flexibility—since it doesn't require a fixed k, we can stop when clustering is "good enough" based on meaningful metrics

b) Example Scenario:

Scenario: Analyzing customer locations for delivery zone planning.

Why diameter-based stopping is better:

- Suppose we're merging customer locations into delivery zones
- We have a constraint that any delivery zone cannot exceed 10 km in diameter (maximum distance between any two customers in the zone)
- We should stop merging clusters when the next merge would create a cluster with diameter > 10 km
- This is more meaningful than stopping at an arbitrary number like "5 zones" because:
    - 5 zones might result in some zones being too large to service efficiently
    - 5 zones might be unnecessarily restrictive if we could efficiently service customers with fewer zones that still meet the diameter constraint

ANS 8.

a)

Step 1: Compute $L_1(10)$ and $L_2(10)$

First, compute $g(x|\mu_i, \sigma_i)$ for each Gaussian:

For Gaussian 1:

$g(10|\mu_1=11, \sigma_1=2) = (1/(2\sqrt{(2\pi)})) \times \exp(-(10-11)^2/(2\times2^2))$
$g(10|11, 2) = (1/(2\sqrt{(2\pi)})) \times \exp(-1/8)$
$g(10|11, 2) = (1/5.013) \times \exp(-0.125)$

$g(10|11, 2) = 0.1995 \times 0.8825 = 0.1761$
For Gaussian 2:

$g(10|\mu_2=51, \sigma_2=2) = (1/(2\sqrt{(2\pi)})) \times \exp(-(10-51)^2/(2\times2^2))$
$g(10|51, 2) = (1/5.013) \times \exp(-1681/8)$
$g(10|51, 2) = 0.1995 \times \exp(-210.125)$
$g(10|51, 2) \approx 0.1995 \times 0 \approx 0$ (essentially zero)
Now compute L_i(x):

$L_1(10) = w_1 \times g(10|11, 2) = 0.5 \times 0.1761 = 0.0881$
$L_2(10) = w_2 \times g(10|51, 2) = 0.5 \times 0 \approx 0$
Step 2: Compute $P_1(10)$ and $P_2(10)$

$\Sigma\square\ L\square(10) = L_1(10) + L_2(10) = 0.0881 + 0 = 0.0881$

$P_1(10) = L_1(10) / \Sigma\square\ L\square(10) = 0.0881 / 0.0881 = 1.0$

$P_2(10) = L_2(10) / \Sigma\square\ L\square(10) = 0 / 0.0881 = 0.0$

Step 3: Interpretation

$P_1(10) = 1.0$ and $P_2(10) = 0.0$ tell us that:

Data point x = 10 has 100% responsibility/posterior probability of belonging to Gaussian 1
Data point x = 10 has 0% responsibility of belonging to Gaussian 2
This makes intuitive sense: x = 10 is very close to $\mu_1 = 11$ (distance of 1) but very far from $\mu_2 = 51$ (distance of 41)
In the E-step, we are essentially assigning this point entirely to Gaussian 1 based on the current parameters

b)

Given:

Data points: x = [10, 12, 50, 52]
|X| = 4
Step 1: Update weight w_1'

$w_1' = (\Sigma_x P_1(x)) / |X|$

$\Sigma_x P_1(x) = P_1(10) + P_1(12) + P_1(50) + P_1(52) = 1.0 + 1.0 + 0.0 + 0.0 = 2.0$

$w_1' = 2.0 / 4 = 0.5$

Step 2: Update mean μ_1'

$\mu_1' = (\Sigma_x\, x \cdot P_1(x)) / (\Sigma_x\, P_1(x))$

Numerator:

$\Sigma_x\, x \cdot P_1(x) = (10 \times 1.0) + (12 \times 1.0) + (50 \times 0.0) + (52 \times 0.0)$
$\Sigma_x\, x \cdot P_1(x) = 10 + 12 + 0 + 0 = 22$
Denominator:

$\Sigma_x\, P_1(x) = 2.0$ (from Step 1)
$\mu_1' = 22 / 2.0 = 11.0$

Step 3: Update variance $\sigma_1'^2$

$\sigma_1'^2 = (\Sigma_x\, P_1(x) \cdot (x - \mu_1')^2) / (\Sigma_x\, P_1(x))$

Using $\mu_1' = 11$:

Numerator:

$\Sigma_x\, P_1(x) \cdot (x - \mu_1')^2 = P_1(10) \cdot (10-11)^2 + P_1(12) \cdot (12-11)^2 + P_1(50) \cdot (50-11)^2 + P_1(52) \cdot (52-11)^2$
$= 1.0 \times (1)^2 + 1.0 \times (1)^2 + 0.0 \times (39)^2 + 0.0 \times (41)^2$
$= 1 + 1 + 0 + 0 = 2$
Denominator:

$\Sigma_x\, P_1(x) = 2.0$
$\sigma_1'^2 = 2 / 2.0 = 1.0$

Therefore: $\sigma_1' = \sqrt{1.0} = 1.0$

Step 4: Parameters for Gaussian 2

By symmetry and using the same formulas:

w_2':

$\Sigma_x\, P_2(x) = 0.0 + 0.0 + 1.0 + 1.0 = 2.0$
$w_2' = 2.0 / 4 = 0.5$
μ_2':

$\Sigma_x\, x \cdot P_2(x) = (10 \times 0.0) + (12 \times 0.0) + (50 \times 1.0) + (52 \times 1.0) = 50 + 52 = 102$
$\mu_2' = 102 / 2.0 = 51.0$
$\sigma_2'^2$:

$\Sigma_x P_2(x)\cdot(x - \mu_2')^2 = 0.0\times(10-51)^2 + 0.0\times(12-51)^2 + 1.0\times(50-51)^2 + 1.0\times(52-51)^2$
$= 0 + 0 + 1 + 1 = 2$
$\sigma_2'^2 = 2 / 2.0 = 1.0$
$\sigma_2' = 1.0$
Rewriting updated parameters:

Gaussian 1: $w_1' = 0.5$, $\mu_1' = 11.0$, $\sigma_1' = 1.0$
Gaussian 2: $w_2' = 0.5$, $\mu_2' = 51.0$, $\sigma_2' = 1.0$

c)

i) Why GMM parameter estimation is hard:

The problem is hard because the assignment of data in the mixture is not known. Specifically:

We have data points but don't know which Gaussian distribution each point belongs to
For example, in histogram-based applications, we do not know which histogram bin belongs to which Gaussian distribution
Without knowing the assignments, we cannot directly compute the parameters
Without knowing the parameters, we cannot determine the assignments

ii) E-Step (Expectation Step):

Assumes: We know the parameters (w_1, μ_1, σ_1, μ_2, σ_2)

Computes: The assignment or responsibility of each point - specifically, how likely each point is to belong to each of the Gaussian distributions

Mathematically:

Computes $L_i(x) = w_i \times g(x|\mu_i, \sigma_i)$ for each Gaussian i
Computes $P_i(x) = L_i(x) / \Sigma\square\, L\square(x)$, which represents the posterior probability that point x belongs to Gaussian i
This answers: "Given the current parameters, what is the probability that this point came from Gaussian 1 vs Gaussian 2?"

iii) M-Step (Maximization Step):

Assumes: We know the assignment/contribution of each point to each distribution (from the E-step)

Computes: Updates the parameters of each distribution based on the weighted contributions

Specifically updates:

Weight: w'_i (proportion of data belonging to this Gaussian)
Mean: μ'_i (weighted average of points assigned to this Gaussian)
Variance: σ'^2_i (weighted variance of points assigned to this Gaussian)

iv) Convergence to local maxima and initialization:

What it means:

The EM algorithm provides the Maximum Likelihood Estimate (MLE) of the parameters
However, the method is iterative and may converge to local maxima rather than the global maximum
This means the final solution depends on where we start (the initial parameters)
Importance of initialization:

Different initializations (w_1, μ_1, σ_1, μ_2, σ_2) can lead to different final solutions
Poor initialization might lead to a suboptimal local maximum
Good initialization strategies include:

- Random initialization with multiple restarts
- K-means initialization
- Using domain knowledge to set initial means

The algorithm will converge to some maximum, but not necessarily the best maximum